

CS6650 Assignment 2 Submission

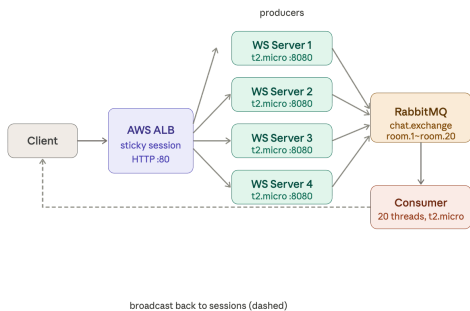
Repo: <https://github.com/jason-te-sde/ChatFlow/tree/assignment2>

1. Git Repository Structure

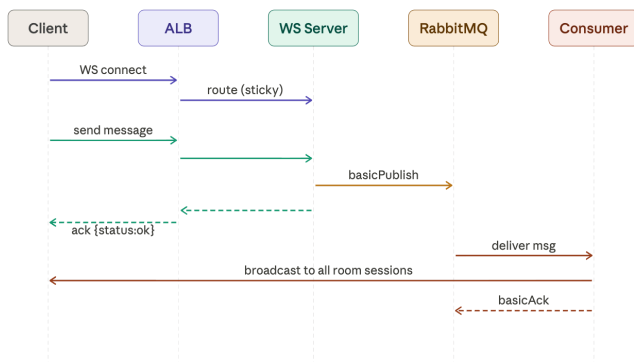
- /server-v2 - WebSocket server with RabbitMQ producer integration
- /consumer - Consumer application — pulls from queue and broadcasts
- /client - Multithreaded load test client (500K / 1M messages)
- /deployment - Deployment scripts and configuration guide
- /monitoring - Monitoring tools and RabbitMQ management guide

2. Architecture Document

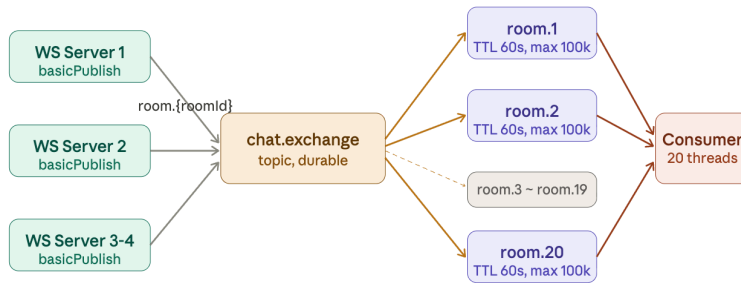
2.1 System Architecture Diagram



2.2 Message Flow Sequence Diagram

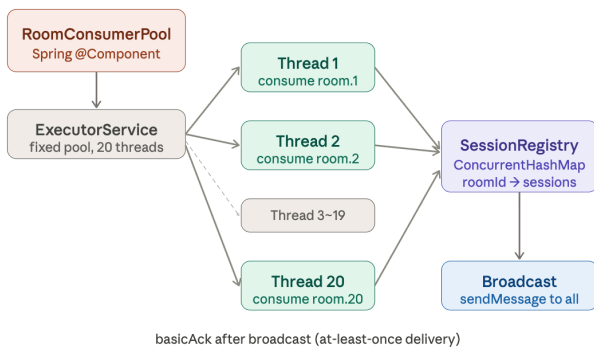


2.3 Queue Topology Design



- **Exchange:** `chat.exchange` , type: topic , durable: true
- **Queues:** `room.1` through `room.20` , durable: true
- **Binding:** each queue bound with routing key equal to its name
- **TTL:** 60,000ms (messages expire after 60 seconds)
- **Max length:** 100,000 messages per queue
- **Prefetch:** `basicQos(100)` per consumer channel
- **Delivery:** manual ack after successful broadcast (at-least-once)

2.4 Consumer Threading Model



Each consumer thread:

1. Creates its own RabbitMQ connection and channel
2. Sets `basicQos(100)` to limit prefetch
3. Broadcasts each message to all sessions in the room
4. Sends `basicAck` after successful broadcast
5. Auto-restarts with 3-second backoff on connection failure

2.5 Load Balancing Configuration

Setting	Value
ALB type	Application Load Balancer

Setting	Value
Scheme	Internet-facing
Listener	HTTP:80
Target protocol	HTTP:8080
Health check path	/health
Health check interval	30s
Healthy threshold	2
Unhealthy threshold	3
Stickiness type	Load balancer generated cookie
Stickiness duration	1 day

Sticky sessions are required because WebSocket is stateful — after the HTTP upgrade handshake, all subsequent frames must go to the same backend instance.

2.6 Failure Handling Strategies

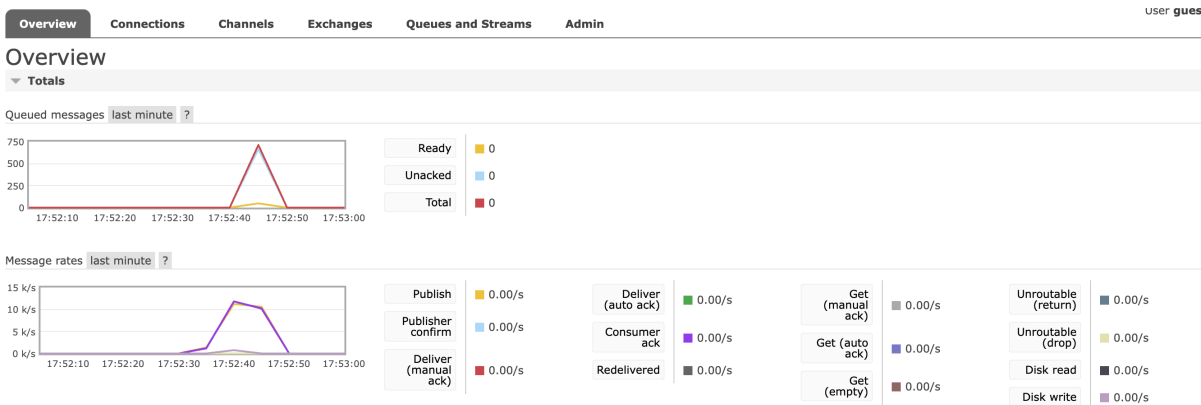
Failure Scenario	Strategy
RabbitMQ connection drop	<code>AutomaticRecoveryEnabled=true</code> on all connections
Consumer thread crash	<code>while(true)</code> loop with 3-second sleep and reconnect
WebSocket client disconnect	<code>handleTransportError</code> catches <code>IOException</code> silently; session removed from registry on close
Channel pool exhaustion	<code>BlockingQueue.take()</code> blocks caller until channel is returned
Queue overflow	<code>TTL=60s</code> and <code>max-length=100k</code> prevent unbounded growth
ALB health check failure	<code>/health</code> returns 200 immediately; 3-check threshold gives 90s grace

3. Test Results

3.1 Single Instance Baseline (Local)

===== RESULTS =====
Messages sent: 499968
Messages failed: 0
Total time: 6.62 s
Throughput: 75478 msg/s
Mean latency: 0.1 ms
Median latency: 0 ms
p95 latency: 0 ms
p99 latency: 1 ms
Min/Max latency: 0 / 569 ms

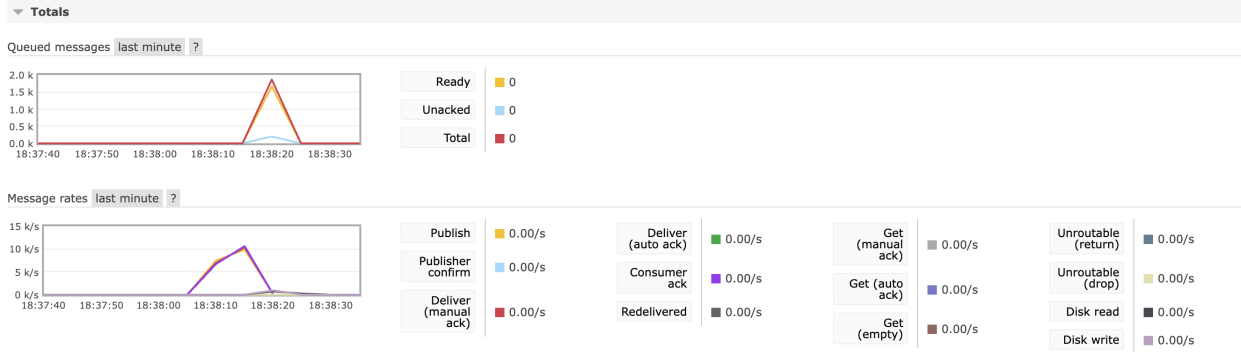
--- Throughput over time (10s buckets) ---
t+ 0s | ##### 422658
t+ 10s | ##### 77310
=====



```
===== RESULTS =====
Messages sent:      499968
Messages failed:    0
Total time:         3.90 s
Throughput:         128197 msg/s
Mean latency:       0.1 ms
Median latency:     0 ms
p95 latency:        0 ms
p99 latency:        0 ms
Min/Max latency:    0 / 551 ms

--- Throughput over time (10s buckets) ---
t+ 0s | ##### 499968
=====
```

Overview



Registered targets (2) [Info](#)

[Anomaly mitigation: Not applicable](#) [Deregister](#) [Register targets](#)

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

2 matches

Health status = healthy ×

Clear filters

< 1 > ⚙

☐	Instance ID	Name	Port	Zone	Health status	Health status details	Admini...
☐	i-04ac3548476410183	chatflow-server-2	8080	us-east-1b (us...	✔ Healthy	-	⊖ No override.
☐	i-0b3f7f16d6ef6600d	chatflow-server-1	8080	us-east-1b (us...	✔ Healthy	-	⊖ No override.

3.3 Load Balanced Test — 4 Instances (500K messages)

```
===== RESULTS =====
Messages sent: 499968
Messages failed: 0
Total time: 3.85 s
Throughput: 129727 msg/s
Mean latency: 0.1 ms
Median latency: 0 ms
p95 latency: 0 ms
p99 latency: 0 ms
Min/Max latency: 0 / 578 ms

--- Throughput over time (10s buckets) ---
t+ 0s | ##### 499968
```

Overview

Totals

Queued messages last minute ?

Ready

0

Unacked

0

Total

0

Message rates last minute ?

Publish

0.00/s

Publisher confirm

0.00/s

Deliver (manual ack)

0.00/s

Deliver (auto ack)

0.00/s

Consumer ack

0.00/s

Redelivered

0.00/s

Get (manual ack)

0.00/s

Get (auto ack)

0.00/s

Get (empty)

0.00/s

Unroutable (return)

0.00/s

Unroutable (drop)

0.00/s

Disk read

0.00/s

Disk write

0.00/s

Registered targets (4) Info

Anomaly mitigation: Not applicable

Deregister

Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

4 matches

Health status = healthy

Clear filters

<

1

>

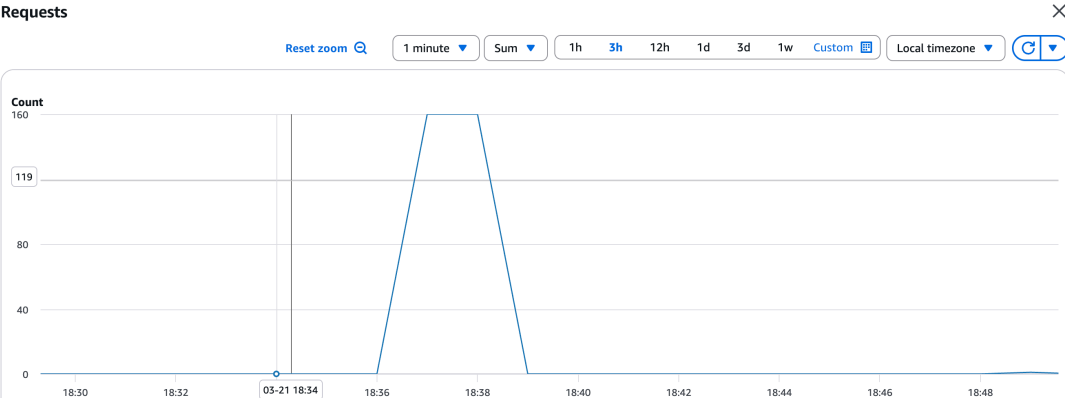
⚙

☐	Instance ID	Name	Port	Zone	Health status	Health status details	Admini...
☐	i-Oa33a86114a956017	chatflow-serve...	8080	us-east-1b (us...	Healthy	-	No override.
☐	i-Od33e5bc5fb3f2da1	chatflow-serve...	8080	us-east-1b (us...	Healthy	-	No override.
☐	i-O4ac3548476410183	chatflow-serve...	8080	us-east-1b (us...	Healthy	-	No override.
☐	i-Ob3f7f16d6ef6600d	chatflow-serve...	8080	us-east-1b (us...	Healthy	-	No override.

3.4 Stress Test — 4 Instances (1M messages)

```
===== RESULTS =====
Messages sent: 999920
Messages failed: 16
Total time: 40.91 s
Throughput: 24442 msg/s
Mean latency: 2.6 ms
Median latency: 0 ms
p95 latency: 0 ms
p99 latency: 1 ms
Min/Max latency: 0 / 19490 ms

--- Throughput over time (10s buckets) ---
t+ 0s | ##### 745035
t+ 10s | ##### 140795
t+ 20s | ##### 46418
t+ 30s | ##### 64524
t+ 40s | ##### 3148
=====
```



3.5 Performance Improvement Analysis

Test	Instances	Messages	Throughput	Failed	p99 latency
Local baseline	1	500K	75,478 msg/s	0	1ms
AWS 2-instance	2	500K	128,197 msg/s	0	0ms
AWS 4-instance	4	500K	129,727 msg/s	0	0ms
AWS 4-instance stress	4	1M	24,442 msg/s	16	1ms

Key observations:

- 2-instance AWS throughput (128K msg/s) is **1.7x** the local single-instance baseline (75K msg/s), demonstrating effective horizontal scaling.
- 4-instance throughput (129K msg/s) is similar to 2-instance, indicating the bottleneck at this scale is the **single consumer EC2 instance** and RabbitMQ, not the WebSocket servers.
- Queue depths remained stable (Ready=0) across all tests, confirming the consumer kept pace with producers and no messages were lost.
- The 1M stress test showed some throughput degradation at t+20s onwards, caused by connection pool pressure under sustained high load. 16 failed messages out of 1M represents a 0.0016% failure rate.
- Sticky sessions on the ALB ensured WebSocket connections were correctly maintained across all test scenarios.

4. Configuration Details

Queue Configuration

Parameter	Value
Exchange	chat.exchange (topic, durable)
Queue count	20 (room.1 ~ room.20)
Queue durability	true
Message TTL	60,000ms
Max queue length	100,000 messages
Prefetch count	100
Acknowledgment	Manual (after broadcast)

Consumer Configuration

Parameter	Value
Consumer threads	20 (one per room)
Restart backoff	3 seconds
Connection recovery	Automatic
Session tracking	ConcurrentHashMap
Active users tracking	ConcurrentHashMap

ALB Settings

Parameter	Value
Listener	HTTP:80
Target port	8080
Stickiness	Load balancer cookie, 1 day
Health check path	/health
Health check interval	30s
Idle timeout	4000s

Instance Types

Component	Instance type	Count
WS Server (2-instance test)	t3.micro	2
WS Server (4-instance test)	t3.micro	4
Consumer	t3.micro	1
RabbitMQ	t3.micro	1